O que falta:

No código em si, o essencial que falta ou está fraco é isto:

Mais importante

1. **Username não está vinculado ao certificado✅**

   O servidor aceita qualquer username no comando register, desde que o cliente tenha algum certificado válido. Falta garantir que o certificado apresentado no handshake pertence ao mesmo username.

   Exemplo de correção esperada: certificado com CN SSI Chat Client - henrique só pode registar/autenticar henrique.
2. **Contactos não são geridos pelo servidor✅**

   O enunciado diz que o servidor deve gerir metadados como listas de contactos. No vosso código, os contactos ficam só no cliente em:

   `contacts/<user>_contacts.json`

   Funciona localmente, mas o servidor não tem comandos tipo add_contact, list_contacts, nem persistência central desses metadados.
3. **Contactos/metadados não têm proteção criptográfica✅**

   As listas de contactos estão em JSON simples. Se a ideia for tratar metadados com rigor de segurança semelhante às mensagens, falta cifrar/assinar esses dados, ou pelo menos protegê-los quando armazenados/alterados.
4. **Registo de utilizadores é demasiado permissivo✅**

   O comando register sobrescreve a chave pública de um username sem validação forte. Um cliente autenticado pode tentar substituir a public key de outro utilizador se usar outro username.

   Isto está ligado ao ponto 1 e é crítico.
5. **Sem verificação forte de identidade na chave pública E2EE✅**

   O cliente pede get_user_key ao servidor e confia na chave recebida. Como o servidor é “honesto mas curioso”, isto pode ser aceitável no modelo básico, mas contra substituição de chaves/MITM seria melhor:
   - associar a chave pública ao certificado;
   - assinar a chave pública com a chave do certificado;

- ou usar fingerprints/verificação manual.

Funcionalidades opcionais do enunciado que não existem

6. **Forward secrecy nas mensagens E2EE**

```
Há forward secrecy no canal cliente-servidor por causa do X25519 efémero,
mas não nas mensagens entre utilizadores. As mensagens usam RSA estático
para encapsular a chave AES. Se a private key do destinatário vazar,
mensagens antigas podem ser decifradas.
```

7. **Mensagens de grupo✅**

   Só há conversas 1-para-1.

8. **Modo descentralizado / PGP-like**

   Toda a comunicação depende do servidor central.

9. **CA dinâmica**

   Já existe CA e certificados, mas não há código para o servidor emitir certificados novos durante o registo. Os certificados estão pré-gerados.

Qualidade/execução

10. **Falta requirements.txt**

```
O projeto depende de cryptography, mas não há ficheiro de dependências.
```

11. **Faltam testes de integração**

    Os testes atuais só testam primitivas criptográficas. Falta testar:

    - handshake real;
    - login com certificado inválido;
    - username diferente do certificado;
    - envio e receção de mensagens;
    - mensagens offline;
    - alteração maliciosa de mensagens assinadas;
    - tentativa de sobrescrever chave pública de outro user.

Se fores corrigir só o que mais pesa para segurança, eu faria nesta ordem:

1. Vincular username ao certificado 2. Impedir overwrite indevido de public keys no register 3. Associar/verificar melhor a public key E2EE do utilizador 4. Adicionar testes de integração 5. Melhorar gestão/proteção de contactos